robots_moviles_25_26

Práctica 3. Programación de tareas en robots móviles

Robots Móviles. Universidad de Alicante. Noviembre 2025

En esta práctica vamos a programar un robot móvil para que realice una tarea estructurada, que implique coordinar una serie de tareas individuales. El objetivo es que sirva para ver conceptos y técnicas que os puedan ser útiles para desarrollar el proyecto de la asignatura durante el último mes de clase.

Tarea a desarrollar

La tarea que debe realizar el robot en esta práctica es muy sencilla y es la siguiente:

1. Inicialmente debe moverse al azar evitando obstáculos hasta que detecte un objeto de color verde en su campo de visión
2. Una vez detectado, el robot debe imprimir un mensaje en la consola y debe volver al punto del que partió inicialmente. Para facilitar la tarea, podéis asumir que el punto de partida es x:-10.0, y:10.0, no es necesario que le pidáis la localización inicial al robot.

En general para desarrollar cualquier tarea necesitaréis saber cómo implementar distintos elementos:

- Subtareas de **movimiento**:
  - Algunas subtareas deben mandar comandos de movimiento al robot (por ejemplo: “navega en línea recta”, “navega al azar evitando obstáculos”.
  - Si tienes un mapa del entorno, algunas subtareas pueden requerir ***navegar* a puntos concretos del mapa** (por ejemplo una tarea de vigilancia). Para eso podéis usar el *stack* de navegación de ROS que ya usásteis en la práctica anterior. En esta se dan algunas pistas de cómo usarlo desde código Python.
- Otras subtareas serán de **detección de condiciones** (por ejemplo, “detectar si estoy en un pasillo”, o “buscar en la imagen de la cámara una pelota de color rojo”). Para estas tendréis que hacer uso de los sensores del robot.
- Necesitaréis **coordinar las subtareas**: por ejemplo habrá subtareas que se deberán realizar en una secuencia (“primero ve al *waypoint* 1 y luego al 2”), otras serán condicionales (“navega aleatoriamente hasta que te encuentres una pelota”), otras tareas serán en paralelo… En robótica para coordinar este tipo de subtareas se pueden usar varios formalismos, como las máquinas de estados finitos y los *behavior trees*.

El entorno para las pruebas

si tenéis ya creado un *workspace* de ROS podéis usarlo, en caso contrario tendréis que crear uno. Puedes copiar el repositorio de la práctica como un *package* dentro de la carpeta `src` del workspace creado:

```
git clone https://github.com/ottocol/practica_conductas_robots_moviles.git
```

y ahora compila el workspace y actualiza las variables de entorno para incluirlo

```
# deberías estar en el directorio base del workspace
colcon build
source install/setup.bash
```

Para poner en marcha el simulador junto con rviz, desde la carpeta del *package* `practica3` :

```
ros2 launch mvsim launch_world.launch.py world_file:=world/prac3.world.xml do_fake
```

Los sensores simulados del robot son un laser que publica en el topic `/scan` y una cámara que publica imágenes RGB en `/camera1_image`

La carpeta `practica3` del *package* contiene los ejemplos que se describirán a continuación y que te pueden servir de base para implementar la práctica.

Procesar las imágenes de la cámara (`color_detector.py`)

Aunque no todas las tareas en robótica móvil requieren vision artificial, es un sensor que puede proporcionar información muy útil. Los turtlebot tienen una cámara Orbbec Astra que nos puede proporcionar información 2D y 3D. Aquí solo veremos cómo procesar la imagen 2D.

OpenCV está integrado con ROS y es la biblioteca que se usa normalmente para procesamiento de imágenes.

En el archivo `color_detector.py` podéis ver el código Python de un nodo de ROS que detecta y cuenta los píxeles de color verde en la imagen captada por la cámara. Si el número de pixeles supera un umbral, publica este número en un determinado *topic*.

Coordinar tareas con máquinas de estados

Para programar un robot que lleve a cabo una tarea compleja resulta útil dividirla en subtareas que habrá que organizar y coordinar. Aunque la coordinación se puede llevar a cabo simplemente con las estructuras de control del lenguaje de programación que estemos usando (bucles, condicionales,…), en general será útil tener algún formalismo que nos permita representar y coordinar tareas. Uno de los más usados en robótica es el de las máquinas de estados.

En ROS2 podemos usar el paquete YASMIN para crear máquinas de estados. Puedes instalarlo con estos comandos (ROS_DISTRO es una variable que deberia tener como valor vuestra distribución de ROS, por ejemplo *jazzy*):

```
sudo apt install ros-$ROS_DISTRO-yasmin ros-$ROS_DISTRO-yasmin-*
```

Para usarlo en los ordenadores del laboratorio tendréis que compilar YASMIN, mirad el apéndice de la práctica.

En el archivo `hola_yasmin.py` puedes ver una máquina de estados sencilla con solo 2 estados, A y B que van cambiando de uno a otro tras 1 segundo cada uno de ellos. Cuando el estado A se ha ejecutado ya 3 veces devuelve un valor que hace que la máquina acabe. Si lo ejecutas verás en la consola cómo va cambiando de estado. Si quieres verlo de forma gráfica puedes usar el comando `ros2 run yasmin_viewer yasmin_viewer_node` . Abre un navegador, escribe la URL `http://localhost:5000` y deberías poder ver gráficamente la máquina de estados.

Cosas a observar en el ejemplo:

- Cómo se definen los estados como clases que heredan de `State` .
  - En el constructor se especifican sus posibles salidas
  - El método `execute` es el cuerpo del estado, en algún momento tiene que devolver una de las salidas posibles
- Todos los métodos de `State` reciben automáticamente como primer parámetro un `Blackboard` , que es un diccionario Python en el que podéis guardar la información que queráis y que se compartirá entre todos los estados. Las claves del diccionario deben ser cadenas. El Blackboard lo inicializamos y se lo pasamos al `execute` de la FSM.
- Mirad en el main cómo se instancian los estados y se “ensambla” la FSM, especificando para cada salida de un estado cuál es el estado siguiente, o si es la salida de la FSM y por tanto ésta se para.

En `patrol_yasmin.py` tenéis cómo definir un estado YASMIN que ejecute una acción en ROS2. La FSM va alternando entre 2 waypoints, (x:0.0, y:0.0) y (x:-10.0, y:-9.0) y navega de uno a otro con una acción de ROS2 de tipo `NavigateToPose` . La FSM navegará indefinidamente entre los 2 waypoints hasta que la paréis con Ctrl-C. Para que funcione este ejemplo tenéis que poner en marcha la navegación:

```
ros2 launch nav2_bringup bringup_launch.py map:=./world/prac3.yaml
```

Y recordad que en rviz2 debéis dar con el ratón una estimación de la posición inicial.

Cosas a observar en el ejemplo:

- Los estados que son acciones de ROS2 heredan de la clase `ActionState`
- En el constructor de la clase llamamos al de la clase base y aquí es donde pasamos los parámetros para configurar la acción:
  - La clase que la implementa (`NavigateToPose`)
  - El nombre de la acción en ROS2 (`/navigate_to_pose`)
  - una función que debe devolver el *goal* de la acción (en nuestro caso un `NavigateToPose.Goal` , que es la posición destino del robot)
  - una función que recibirá *feedback* del *action server*
  - Hay más parámetros, los tenéis en el código fuente de la clase `ActionState` en el repo de YASMIN
- Al constructor de la clase le pasamos el *waypoint* destino (que luego guardamos en la propiedad `wp`) para que la función que construye el *goal* pueda acceder a él.
- No hace falta definir el método `execute` del estado ya que ejecutar el estado es ejecutar la acción ROS2 asociada.
- En el main tenéis, igual que en el otro ejemplo, cómo se ensambla y se ejecuta la máquina.
- Si se pulsa Ctrl-C hay que parar al robot cancelando la navegación al *waypoint* actual. Para cancelar la navegación lo que se hace es guardar en el main un array de estados "cancelables" y cuando se detecta el Ctrl-C (excepción `KeyboardInterrupt`) se llama al `set_cancel` de todos. En realidad solo haría falta cancelar el activo pero saber cuál es nos complica un poco más el código así que los cancelamos todos.

Ayuda para la implementación

Se os deja como ejemplo, el archivo `prac3_base.py` , este estado: "moverse evitando obstáculos y esperando detectar el color verde" (en el código tiene un nombre más corto :)). Así podéis ver cómo encaja el código de publicar/suscribir en ROS dentro de un estado de YASMIN.

Para evitar obstáculos sigue la estrategia simple de mirar los rayos del lidar a -30 y 30 grados y girar siempre en la direccion del más lejano.

> IMPORTANTE: tomad las constantes de las líneas 80-82 como valores iniciales tentativos, ajustadlos para que funcione mejor. También podéis cambiar los ángulos de los rayos para mirar otros ángulos distintos, mirar más rayos, etc.

Baremo y plazo de entrega

- Hasta 5 puntos: implementar correctamente la funcionalidad pedida en el apartado “Tarea a desarrollar”
- Hasta 1 punto: realizar y documentar pruebas de la tarea, indicando si funciona siempre, falla en algún momento, posibles problemas o causas que creéis que tienen los fallos, etc. Documentadla con al menos un video que muestre el funcionamiento.
- Hasta 1 punto: mejorar la tarea añadiendo algún paso adicional, por ejemplo una vez detectado el color, moverse hacia el objeto de color (manteniéndolo centrado en la imagen) hasta llegar a una determinada distancia de él (para la distancia podéis usar el rayo del laser que apunta al frente). Este es solo un ejemplo, podéis inventaros cualquier otro paso.
- Hasta 1 punto: usar información de la cámara de profundidad para detectar a qué distancia está el objeto detectado y crear un `Marker` en rviz con la posición en el mundo (para esta posición habrá que hacer un cálculo con la posición actual del robot, la dirección en que mira y la distancia a la que está el objeto).
- Hasta 2 puntos: implementar la funcionalidad pedida en el apartado “Tarea a desarrollar” usando *behavior trees* además de máquinas de estados. Para ello podéis usar la bibliotecas py_trees_ros o cualquier otra que encontréis por Internet.

La práctica se podrá entregar hasta el ~~domingo 23 de noviembre~~ **domingo 30 de noviembre a las 23:59**

Apéndice: compilar YASMIN en los ordenadores del laboratorio

```
#instalar dependencias de YASMIN en un entorno virtual python y activarlo
python3 -m venv --system-site-packages ~/ros2_venv
source ~/ros2_venv/bin/activate
pip install expiringdict flask waitress
source ~/ros2_venv/bin/activate

#bajarse el código fuente de YASMIN y compilarlo
mkdir -p ~/ros2_yasmin_ws/src
cd ~/ros2_yasmin_ws/src
git clone --branch 3.5.1 https://github.com/uleroboticsgroup/yasmin.git
cd ~/ros2_yasmin_ws
source /opt/ros/jazzy/setup.bash
colcon build
source ~/ros2_yasmin_ws/install/setup.bash
```